Inginerul Cu AI

Vibe Coding

20 de Prompturi Profesionale AI

Începe Acum →

AI

Inginerul Cu AI

Ghid de Utilizare

20 de Prompturi Profesionale AI
Chat GPT - Gemini - Claude

CUM SE FOLOSEȘTE:

1. Selectează promptul potrivit obiectivului tău.

2. Fiecare prompt conține 2 slide-uri:

- Slide 1: Setup & Obiectiv
- Slide 2: Livrabil final

3. Personalizează [variabilele din paranteze].

4. Copiază textul în platforma AI preferată.

5. Rafinează prin iterații.

✦ **Fiecare prompt arată când să îl folosești.**

AI

Inginerul Cu AI

❖ #01

Generarea unei Aplicații Web de la Zero (End-to-End Build)

Când să folosești: Atunci când intenționezi să dezvolți o aplicație web complet funcțională, pornind de la un concept brut, fără a scrie manual cod sursă.

✦ **ACȚIONEAZĂ CA:** Un Inginer Software Senior (Staff Engineer) și Arhitect Principal de Sistem (Vibe Coder), expert în dezvoltarea rapidă de aplicații (RAD) folosind stive tehnologice moderne.

⬢ **OBIECTIVUL TĂU:** Construiește arhitectura completă și scrie codul sursă integral pentru o nouă aplicație web: [descrierea detaliată a aplicației tale]. Obiectivul tău este să livrezi un produs viabil, complet funcțional și gata de producție (Production-ready).

❖ #01

Generarea unei Aplicații Web de la Zero (End-to-End Build)

Când să folosești: Atunci când intenționezi să dezvolți o aplicație web complet funcțională, pornind de la un concept brut, fără a scrie manual cod sursă.

› LIVRABIL EXECUTAT:
Structurează proiectul și generează codul final respectând strict următorii parametri de dezvoltare:

1. Stiva Tehnologică (Tech Stack):
- Limbaj și Framework principale: [ex: HTML/CSS/JS nativ, React, Next.js sau Vue]

2. Funcționalități Esențiale (Core Features):
- [Descrie Prima Funcționalitate - ex: Sistem de creare cont utilizator]
- [Descrie A Doua Funcționalitate - ex: Panou de bord interactiv]
- [Descrie A Treia Funcționalitate - ex: Procesare de plăți externe]

3. Arhitectura și Designul Vizual:
- Estetica interfeței (UI): [Minimalist / Modern / Corporativ / Vibrant]
- Adaptabilitate (Responsiveness): Implementare nativă pentru dispozitive mobile și desktop
- Baza de date și Stocare: [Stocare Locală (localStorage) / Firebase / Supabase / Fără stocare]
- Sistem de Autentificare: [Integrat (Da) / Fără autentificare (Nu)]

❖ GÂNDIRE PAS CU PAS: Acționează ca un dezvoltator asertiv. Generează codul cap-coadă, absolut complet și executabil. Sub nicio formă nu folosi substituenți de cod (placeholders) de tipul "pune logica ta aici". La finalizarea codului, oferă un rezumat tehnic concis de 3-5 rânduri explicând structura arhitecturală a fișierelor și librăriile externe injectate.

❖ #02

Depanare și Soluționarea Erorilor (Debugging & Error Resolution)

Când să folosești: Atunci când execuția codului eșuează, aplicația se blochează (Crash) sau terminalul returnează o eroare critică pe care nu o poți descifra.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Securitate și Expert în Depanare (Senior Debugger), recunoscut pentru capacitatea chirurgicală de a izola, explica și neutraliza cele mai complexe erori tehnice de sistem.

⬢ **OBIECTIVUL TĂU:** Diagnostichează și remediază următoarea eroare critică apărută în timpul execuției. Trebuie să oferi nu doar codul reparat, ci și o înțelegere clară a vulnerabilității care a provocat incidentul.

❖ #02

Depanare și Soluționarea Erorilor (Debugging & Error Resolution)

Când să folosești: Atunci când execuția codului eșuează, aplicația se blochează (Crash) sau terminalul returnează o eroare critică pe care nu o poți descifra.

› LIVRABIL EXECUTAT:

Structurează soluția de depanare parcurgând analitic acești pași de reparare:

1. Datele Erorii Curente:
- EROAREA PRECISĂ: [Lipește aici mesajul exact returnat de terminal/browser]
- CODUL VULNERABIL: [Lipește aici secțiunea de cod problematică]
- CONTEXTUL APLICAȚIEI: [Explică scurt ce intenționai să facă programul la acel pas]
- Mediul de Dezvoltare (Tech Stack): [Limbajul și framework-ul utilizat]

2. Protocolul de Remediere (Patch):
- Izolarea Cauzei: Identifică instantaneu sursa exactă a blocajului tehnic.
- Explorarea Cauzalității: Explică DE CE a eșuat codul, folosind o analogie simplă.
- Codul Curat: Furnizează exclusiv secțiunea de cod corectată, integrabilă direct prin Copy/Paste.
- Prevenție Sistemică: Recomandă o metodă defensivă (ex: TypeScript, clauze Try/Catch) pentru eradicarea totală a acestei familii de erori pe viitor.

❖ GÂNDIRE PAS CU PAS: Nu te pierde în scuze sau justificări inutile. Tratează eroarea obiectiv, expune slăbiciunea logicii inițiale și livrează soluția de cod impecabilă, optimizând totodată performanța rutei afectate, dacă soluția o cere.

❖ #03

Refactorizarea și Optimizarea Codului (Code Refactoring)

Când să folosești: Atunci când ai la dispoziție o bază de cod funcțională, dar care execută lent, este greu de citit sau prezintă vulnerabilități și erori potențiale de logică.

✦ **ACȚIONEAZĂ CA:** Un Inginer Software Principal (Principal Software Engineer) obsedat de arhitectura codului curat (Clean Code) și de eficiența algoritmică, specializat în revizuirea de cod (Code Review) la nivel de corporație.

⬢ **OBIECTIVUL TĂU:** Refactorizează și restructurează complet următorul bloc de cod, ridicându-l la un standard de excelență tehnică. Codul trebuie să ruleze perfect, consumând minimul absolut de resurse (memorie și timp de execuție).

❖ #03

Refactorizarea și Optimizarea Codului (Code Refactoring)

Când să folosești: Atunci când ai la dispoziție o bază de cod funcțională, dar care execută lent, este greu de citit sau prezintă vulnerabilități și erori potențiale de logică.

› LIVRABIL EXECUTAT:

Structurează revizuirea codului conform următoarelor standarde de performanță:

1. Codul de Referință (Sursa de refactorizat):
- [Lipește aici codul tău sursă complet]

2. Obiective Principale de Intervenție:
- Îmbunătățește drastic lizibilitatea codului (utilizând convenții clare de denumire)
- Optimizează performanța și complexitatea algoritmică (Time/Space Complexity)
- Documentează codul injectând comentarii scurte și utile (Doctype/Inline)

3. Constrângeri de Execuție:
- Păstrează **exact** aceeași funcționalitate finală a aplicației (fără adăugiri laterale)
- Explică logic fiecare modificare arhitecturală majoră, justificând-o
- Identifică și remediază proactiv orice bug ascuns sau vulnerabilitate de securitate
- Sugerează și implementează tipare de design (Design Patterns) consacrate, dacă structura o permite.

4. Nivelul de Complexitate Vizat (Target):
- [Pentru Începători (Explicat detaliat) / Intermediar / Standard de Producție (Production-grade)]

❖ GÂNDIRE PAS CU PAS: Joacă rolul unui mentor strict. Codul pe care mi-l vei returna trebuie să fie impecabil vizual și tehnic. Analizează fiecare structură repetitivă și extrage-o în funcții pure. Demonstrează excelența.

❖ #04

Crearea de Componente UI Reutilizabile (Reusable UI Components)

Când să folosești: Atunci când ai nevoie să dezvolți o componentă de interfață grafică (UI) robustă, care va fi multiplicată și refolosită în diferite rute sau ecrane ale aplicației tale.

✦ **ACȚIONEAZĂ CA:** Un Inginer Front-End Senior (Senior Front-End Engineer) și Arhitect de Sisteme de Design (Design System Architect), expert în construirea de biblioteci de componente modulare și ultra-scalabile.

⬢ **OBIECTIVUL TĂU:** Proiectează și generează codul complet pentru o componentă UI izolată și 100% reutilizabilă, destinată funcționalității de [tip: Fereastră Modală (Modal) / Card de Produs (Card) / Bară de Navigație (Navbar) / Formular].

❖ #04

Crearea de Componente UI Reutilizabile (Reusable UI Components)

Când să folosești: Atunci când ai nevoie să dezvolți o componentă de interfață grafică (UI) robustă, care va fi multiplicată și refolosită în diferite rute sau ecrane ale aplicației tale.

› LIVRABIL EXECUTAT:

Structurează componenta grafică impunând următoarele specificații tehnice stricte:

1. Mediul de Dezvoltare Front-End:
- Framework vizat: [React / Vue / Svelte / Vanilla JS]
- Motorul de Stilizare vizat: [Tailwind CSS / CSS Modules / SCSS / Styled Components]
- Tema vizuală aplicată: [Întunecată (Dark) / Luminoasă (Light) / Personalizată (Custom)]

2. Parametrii și Stările Componentei (Props & States):
- Proprietate (Prop) 1: [ex: titluText]
- Proprietate (Prop) 2: [ex: onClickHandler]
- Include tratarea vizuală pentru toate stările de interacțiune: Implicită (Default), Survol (Hover), Activată (Active), Dezactivată (Disabled).

3. Cerințe de Standard și Accesibilitate:
- Implementează standarde absolute de accesibilitate: Etichete ARIA (ARIA labels) și navigare exclusiv din tastatură (Keyboard nav).
- Definește și exportă clar toate tipurile de date folosind TypeScript (TypeScript Interfaces).
- Asigură-te că designul este complet adaptabil (Mobile-first responsive).

❖ GÂNDIRE PAS CU PAS: O componentă reutilizabilă proastă distruge performanța aplicației la scară largă. Scrie codul modular, expunând clar interfața API (Props) pe care alte rute o vor apela. Finalizează cu un cod-exemplu concis de implementare arătând exact cum trebuie importată și utilizată componenta creată.

❖ #05

Integrare API REST Comunicațională (REST API Integration)

Când să folosești: Atunci când dorești să conectezi interfața aplicației tale la baze de date externe sau servicii terțe (Third-Party Services) pentru a prelua date în timp real.

✦ **ACȚIONEAZĂ CA:** Un Inginer Backend Senior (Senior Backend Engineer) și Arhitect de Date, expert în manipularea asincronă a pachetelor de date și securizarea fluxurilor de comunicație prin API.

⬢ **OBIECTIVUL TĂU:** Implementează și documentează o conexiune de rețea ultra-sigură la API-ul extern: [numele API-ului țintă]. Obiectivul tău este preluarea datelor fără nicio întrerupere de serviciu (Zero Downtime) și administrarea meticuloasă a erorilor de rețea.

❖ #05

Integrare API REST Comunicațională (REST API Integration)

Când să folosești: Atunci când dorești să conectezi interfața aplicației tale la baze de date externe sau servicii terțe (Third-Party Services) pentru a prelua date în timp real.

› LIVRABIL EXECUTAT:
Structurează blocul de cod pentru integrarea API respectând acest protocol strict:

1. Parametrii Tehnici ai Cererii (Request Configuration):
- Punctul final de acces (Endpoint): [Inserează URL-ul exact]
- Metoda de transport HTTP: [Alege exclusiv: GET / POST / PUT / PATCH / DELETE]
- Protocolul de Autentificare: [Cheie API (API Key) / Jeton de purtător (Bearer Token) / Fără]

2. Structura Datelor Tranzitate (Data Payload):
- Definește cu precizie utilajul de date vizat: [Specifică exact ce date trimiți sau extragi]

3. Arhitectura Funcțională a Codului:
- Scrie o funcție asincronă non-blocantă utilizând exclusiv sintaxa `async/await`.
- Implantează un scut defensiv de captare a erorilor: structură completă `try/catch`.
- Gestionează Interfața Grafică (UI) expunând clar starea de "Încărcare" (Loading state) și "Eroare" (Error state).
- Scrie logica de parsare (descompunere) a JSON-ului și afișarea datelor finalizate.

4. Măsuri de Securitate Obligatorii:
- Stochează și apelează cheile de acces API exclusiv prin variabile de mediu izolate (ex: `.env`).

❖ GÂNDIRE PAS CU PAS: O rețea este prin definiție failibilă. Codul tău trebuie să presupună, prin design, că serverul extern va pica ocazional. Scrie mecanismul de captare a erorilor astfel încât aplicația clientului să rămână grafic funcțională chiar dacă API-ul returnează codul statut 500.

Sample JSON model pentru maparea datelor:
[Lipește aici pur și simplu un fragment din răspunsul JSON dorit, dacă este disponibil]

❖ #06

Generarea unei Pagini de Prezentare (Full Landing Page)

Când să folosești: Atunci când trebuie să lansezi rapid o pagină de destinație comercială (Landing Page) destinată exclusiv conversiei vizitatorilor în clienți, folosind un design modern.

✦ **ACȚIONEAZĂ CA:** Un Dezvoltator Front-End Full-Stack și Designer UX/UI, recunoscut pentru crearea de pagini de vânzare (Sales Pages) de înaltă conversie, extrem de rapide la încărcare.

⬢ **OBIECTIVUL TĂU:** Generează codul sursă integral pentru o pagină de destinație (Landing Page) premium, complet funcțională, pentru [specifică produsul sau serviciul tău]. Structura vizuală trebuie să direcționeze obsesiv utilizatorul către butonul Call-To-Action.

❖ #06

Generarea unei Pagini de Prezentare (Full Landing Page)

Când să folosești: Atunci când trebuie să lansezi rapid o pagină de destinație comercială (Landing Page) destinată exclusiv conversiei vizitatorilor în clienți, folosind un design modern.

› LIVRABIL EXECUTAT:
Structurează fișierul codului respectând următoarele module vizuale și tehnice obligatorii:

1. Poziționarea Mărcii:
- Profilul publicului țintă: [Descrie exact cine accesează pagina]
- Tonul mesajului comercial: [Profesional / Casual / Energic / Luxos]
- Paleta cromatică a brandului: [Specifică exact culorile HEX sau deleagă alegerea către AI]

2. Arhitectura Secțiunilor Externe (Scroling Journey):
- Secțiunea Erou (Hero Section) cap de afiș, cu Titlu imposibil de ignorat (Headline) și Buton de Acțiune clar (CTA).
- Structură de Beneficii, expunând 3-6 rezultate finale atractive pentru client.
- Modul de Validare Socială (Social Proof): spațiu pentru mărturii și testimoniale.
- Secțiunea Tabelului de Prețuri (Pricing Table) cu diferențiatori clari între pachete.
- Modul de Întrebări Frecvente (FAQ) colapsabil cu 5 întrebări standard.
- Secțiunea de Subsol (Footer navigabil).

3. Cerințe Tehnice Stricte:
- Codare Mono-fișier: Injectează totul într-un unic fișier (HTML combinat cu CSS și JS intern) pentru portabilitate instantanee.
- Aspect Fluid (Responsive-First): Arhitectura trebuie să se caleze perfect pe rezoluții mobile, tablete și desktop în mod nativ.
- Interactivitate Micro-UI: Adaugă tranziții vizuale și animații subtile la derularea paginii (Scroll animations) pentru un design premium.

❖ GÂNDIRE PAS CU PAS: Nu programa o pagină banală tip panou de afișaj. Un Landing Page excelent pune la dispoziție o narațiune vizuală. Scrie codul UI cu padding-uri mari specifice designului modern occidental, fonduri asimetrice și contraste puternice de text pentru 100% lizibilitate.

❖ #07

Formular cu Validare Client-Side Completă (Form Validation)

Când să folosești: Atunci când ai nevoie de un formular complex de captare a datelor (Contact, Înregistrare sau Checkout) care previne introducerea informațiilor eronate înainte de a interoga serverul.

✦ **ACȚIONEAZĂ CA:** Un Inginer Front-End (Front-End Engineer) meticulos, specializat în Arhitectura Experienței Utilizatorului (UX) și validarea datelor pe partea de client (Client-side validation).

⬢ **OBIECTIVUL TĂU:** Generează codul complet și securizat pentru un formular web de tip [specifică tipul: Contact / Creare Cont / Plată]. Formularul trebuie să prevină activ orice greșeală a utilizatorului, oferind feedback vizual instantaneu și clar.

❖ #07

Formular cu Validare Client-Side Completă (Form Validation)

Când să folosești: Atunci când ai nevoie de un formular complex de captare a datelor (Contact, Înregistrare sau Checkout) care previne introducerea informațiilor eronate înainte de a interoga serverul.

› LIVRABIL EXECUTAT:
Structurează componenta formularului respectând aceste standarde stricte de funcționare:

1. Mediul Tehnologic:
- Framework vizat: [Vanilla JS / React / Vue / Angular]

2. Matricea Câmpurilor și Regulile de Validare (Validation Rules):
- Câmpul 1: [ex: Nume Complet] -> Regula: [Validare format text, minim 3 caractere]
- Câmpul 2: [ex: Adresă Email] -> Regula: [Validare strictă format RFC 5322]
- Câmpul 3: [ex: Număr Telefon] -> Regula: [Validare format internațional + Prefix]
- Identificare vizuală: Toate câmpurile obligatorii vor fi marcate distinct cu asterisc roșu (*).

3. Logica de Securitate a Parolei (Când este cazul):
- Politică de parole aspră: Minim 8 caractere, impunând cel puțin o literă majusculă, un număr și un caracter special.
- Indicator dinamic de putere a parolei (Password Strength Meter) actualizat la fiecare tastare.

4. Arhitectura Experienței de Utilizare (UX Flow):
- Declanșarea validării: Validarea va avea loc exclusiv la evenimentul de părăsire a câmpului (on-blur), nu în timpul tastării (pentru a nu stresa utilizatorul).
- Feedback Erori: Mesaje de eroare textuale explicite plasate fix sub câmpul greșit, însoțite de schimbarea culorii bordurii în roșu.
- Butonul de Trimitere (Submit): Rămâne absolut dezactivat (disabled) până când întregul formular trece de 100% din criteriile de validare.

❖ GÂNDIRE PAS CU PAS: Formularele frustrante distrug rata de conversie. Tratează utilizatorul cu indulgență. Scrie funcțiile de validare codând o logică defensivă solidă, fără a depinde din greșeală de capabilitățile implicite (și adesea limitate) ale browserului web (folosind `novalidate`).

❖ #08

Panou de Bord cu Grafice Interactive (Interactive Dashboard)

Când să folosești: Atunci când trebuie să afișezi date cantitative complexe, convertindu-le într-un format vizual rapid de digerat pentru a extrage insight-uri strategice.

✦ **ACȚIONEAZĂ CA:** Un Inginer Vizualizator de Date (Data Visualization Engineer) și Dezvoltator de Produs, expert în transformarea fișierelor masive de date (Big Data) în interfețe grafice elocvente și decizionale.

⬢ **OBIECTIVUL TĂU:** Construiește codul complet pentru un Panou de Bord (Dashboard) profesional destinat verticalei de [specifică tipul: Analize Financiare / Vânzări e-Commerce / Raportare Performanță].

❖ #08

Panou de Bord cu Grafice Interactive (Interactive Dashboard)

Când să folosești: Atunci când trebuie să afișezi date cantitative complexe, convertindu-le într-un format vizual rapid de digerat pentru a extrage insight-uri strategice.

› LIVRABIL EXECUTAT:

Structurează codul panoului de afișaj respectând aceste canoane tehnologice:

1. Stiva Tehnologică și Librăria Vizuală:
- Librărie de randare grafică: [Chart.js / Recharts / D3.js / ApexCharts]

2. Modelarea Datelor (Data Mocking):
- Folosește următorul pachet de date simulate pentru a popula graficele:
[Lipește aici obiectul JSON sample sau descrie verbal setul de date]

3. Arhitectura Graficelor (Vizualizări Principale):
- Graficul 1: [ex: Grafic de Linier (Line chart) pentru tendința veniturilor lunare]
- Graficul 2: [ex: Grafic Radial (Pie chart) pentru distribuția pe categorii de clienți]

4. Ecosistemul de Funcționalități Analitice:
- Carduri de Performanță Cheie (KPI Cards): Afișează static Metricile Principale (ex: Total, Medie Zilnică, Vârf Maxim) deasupra tuturor graficelor.
- Interacțiune prin Survolare (Hover Tooltips): Afișarea detaliată, la nivel de zecimală, a valorii atunci când cursorul atinge un punct pe grafic.
- Filtrare Temporală Activă: Implementarea unor butoane de comutator (Toggles) funcționale pentru: "Ultimele 7 zile", "30 de zile" și "Anul curent".
- Motor de Export: Un buton asincron pentru exportarea și descărcarea datelor afișate în format CSV.

5. Responsivitate Absolută (Mobile-Ready):
- Aranjarea gridului de grafice (Grid layout) trebuie să devină liniară (o singură coloană) pe ecranele telefoanelor mobile fără tăierea sau suprapunerea etichetelor legendelor.

❖ GÂNDIRE PAS CU PAS: Scopul unui dashboard nu este frumusețea, ci viteza cu care creierul extrage informația. Alege o paletă de culori armonioasă pentru grafice (evitând culorile stridente necorelate) și asigură-te că axele X/Y au scalări matematice coerente.

❖ #09

Sistematizarea Autentificării Securizate (Authentication System)

Când să folosești: Atunci când construiești o aplicație privată și ai nevoie de un zid de securitate impenetrabil pentru înregistrarea (Register) și logarea (Login) utilizatorilor tăi.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Securitate Cibernetică (Cybersecurity Engineer) și Arhitect Backend, specializat în criptografie și protejarea identității digitale a utilizatorilor în aplicații web la scară largă.

⬢ **OBIECTIVUL TĂU:** Implementează arhitectura completă pentru un sistem robust de Autentificare și Autorizare (Auth System). Obiectivul tău este zero breșe de securitate, protejând datele sensibile împotriva oricărui atac cibernetic standard (ex: Brute Force, XSS, CSRF).

❖ #09

Sistematizarea Autentificării Securizate (Authentication System)

Când să folosești: Atunci când construiești o aplicație privată și ai nevoie de un zid de securitate impenetrabil pentru înregistrarea (Register) și logarea (Login) utilizatorilor tăi.

› LIVRABIL EXECUTAT:
Structurează sistemul de identificare respectând următoarele directive de securitate militară:

1. Mediul de Execuție (Tech Stack):
- Tehnologie Backend: [ex: Node.js cu Express / Firebase Auth / Supabase]
- Tehnologie Frontend: [ex: React / Vue / Svelte / Vanilla JS]

2. Fluxul Utilizatorului (User Flow):
- Creare Cont (Register): Cu adresă de email și parolă.
- Accesare Cont (Login) și Delogare securizată (Logout).
- Protecția Rutelor (Route Guards): Redirecționarea automată a persoanelor neautentificate către ecranul de Login.

3. Standarde Obligatorii de Securitate Backend (Security Layer):
- Criptarea ireversibilă a parolelor în baza de date folosind algoritmi puternici (ex: `bcrypt`).
- Administrarea sesiunilor exclusiv prin Jetoane Web JSON (JWT Tokens) sau Sesiuni de Server (Session-based auth) cu stocare securizată (HttpOnly Cookies).
- Limitarea ratei de interogare (Rate Limiting) pentru Login: Maxim 5 încercări eșuate pentru a bloca atacurile Brute Force.

4. Funcționalități Premium (Extra):
- Integrarea fluxului de "Resetare Parolă" securizat prin link unic de resetare trimis pe email.
- Autentificare Socială (SSO): Opțiune de conectare cu [Google / Apple / GitHub] prin OAuth 2.0.
- Persistența sesiunii: Funcția reflexă "Ține-mă minte" (Remember me) validă 30 de zile.

❖ GÂNDIRE PAS CU PAS: Nu există scurtături în securitate. Nu trimite niciodată parole în format text clar (Plaintext). Construiește acest sistem presupunând că aplicația ta ar putea stoca datele financiare ale clienților. Generează întregul flux de la interceptarea rutei în Frontend până la validarea JWT-ului în Backend.

❖ #10

Construcția unei Aplicații CRUD Locale (CRUD App with LocalStorage)

Când să folosești: Atunci când dorești să lansezi instantaneu un prototip funcțional sau o aplicație utilitară care stochează date direct în browserul utilizatorului, având zero costuri de server.

✦ **ACȚIONEAZĂ CA:** Un Inginer Full-Stack (Full-Stack Engineer) axat pe productivitate extremă și Prototoipare Rapidă (Rapid Prototyping), capabil să livreze aplicații complet funcționale fără a depinde de o bază de date externă.

⬢ **OBIECTIVUL TĂU:** Creează arhitectura și codul sursă complet pentru o aplicație web complet funcțională bazată pe stocarea locală a browserului (`localStorage`). Aplicația va fi dedicată managementului de [specifică tema: Sarcini Zilnice (Tasks) / Rețete Culinare / Agendă Contacte / Notițe Personale].

❖ #10

Construcția unei Aplicații CRUD Locale (CRUD App with LocalStorage)

Când să folosești: Atunci când dorești să lansezi instantaneu un prototip funcțional sau o aplicație utilitară care stochează date direct în browserul utilizatorului, având zero costuri de server.

› LIVRABIL EXECUTAT:
Structurează aplicația respectând perfect ciclul fundamental al gestionării datelor:

1. Nucleul Operațional (Cele 4 Operații CRUD):
- Creare (Create): Formular curat cu validare pentru adăugarea unei noi înregistrări.
- Citire (Read): Interfață de afișare pentru listarea tuturor înregistrărilor salvate.
- Actualizare (Update): Metodologie de editare (direct în rând sau printr-o fereastră Modală).
- Ștergere (Delete): Eliminarea unei intrări, condiționată strict de un mesaj de Confirmare (pentru prevenirea ștergerilor accidentale).

2. Motorul de Persistență (State Management):
- Salvarea și extragerea imediată și asincronă a pachetelor de date (Stringify/Parse JSON) exclusiv prin API-ul `localStorage` al browserului.

3. Sistemul de Design Visual:
- Structura interfeței (Layout): [ex: Grilă de Cartonașe (Card Grid) / Tabel Analitic / Listă Verticală]
- Cromatica (Theme): [ex: Întunecată (Dark Mode) / Luminoasă (Light Mode) / Alternativă]

4. Funcționalități Utilitare Avansate (Extra Feature Set):
- Motor de Căutare (Search) internă cu filtrare instantanee în timp real a datelor.
- Sortare alfabetică sau cronologică a listei apăsând click pe [Specifică câmpul de sortare].
- Afișarea în timp real a unui Contor (Counter) cu numărul total de înregistrări active.
- Un buton executabil pentru a exporta totul într-un fișier descărcabil de tip `.json`.

❖ GÂNDIRE PAS CU PAS: Succesul unei aplicații bazate pe LocalStorage este izolarea excepțională a logicii. Scrie un utilitar/o clasă separată doar pentru operațiunile cu stocarea locală (get/set/remove), pe care componenta UI doar să le apeleze. Construiește cod curat (Clean Code) și o estetică minimalistă (Minimalist UI).

❖ #11

Arhitectura unui Chatbot Inteligent (AI Chatbot Integration)

Când să folosești: Atunci când intenționezi să integrezi o inteligență artificială conversațională direct pe platforma ta, oferind asistență continuă (24/7) fără intervenție umană.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Inteligență Artificială (AI Engineer) și Dezvoltator Interfețe Conversaționale (Conversational UI Developer), expert în implementarea modelelor mari de limbaj (LLMs) în aplicații comerciale.

⬢ **OBIECTIVUL TĂU:** Construiește codul complet pentru interfața și logica de backend a unui Asistent Virtual (Chatbot), integrat nativ cu API-ul: [OpenAI / Claude / Gemini API]. Obiectivul este crearea unei conversații fluide, naturale și axate pe [specifică scopul: Suport Clienți / Învățare interactivă / Vânzări indirecte].

❖ #11

Arhitectura unui Chatbot Inteligent (AI Chatbot Integration)

Când să folosești: Atunci când intenționezi să integrezi o inteligență artificială conversațională direct pe platforma ta, oferind asistență continuă (24/7) fără intervenție umană.

› LIVRABIL EXECUTAT:
Structurează sistemul conversațional respectând aceste standarde de proiectare corporativă:

1. Ingineria Interfeței de Chat (Conversational UI):
- Diferențierea vizuală absolută a entităților: Bule de mesaj aliniate la dreapta pentru Utilizator (User) și aliniate la stânga pentru Asistent (AI).
- Modul de Text (Input Field) adaptabil cu suport funcțional pentru tasta `Enter` (Submit pe Enter) și buton clar de "Trimitere".
- Micro-interacțiuni cruciale: Implementează obligatoriu un indicator de tastare animat ("AI-ul scrie...") pentru a masca asincronicitatea răspunsului (Latency).

2. Configurarea Creierului AI (System Prompt):
- Injectează intern următoarea directivă absolută de comportament: "Ești un expert în [domeniul tău]. Răspunzi exclusiv și profesional despre [subiectul ales]. Tonul tău este [educativ / corporativ / prietenos]."

3. Funcționalități Esențiale ale Interfeței (Core Features):
- Memoria Contextuală: Menținerea și stocarea istoricului conversației pentru ca AI-ul să rețină contextul anterior (Conversation History).
- Marcaje Temporale (Timestamps): Afișarea orei exacte și a minutului sub fiecare mesaj trimis (ex: 14:32).
- Export & Resetare: Butoane utilitare esențiale: "Copiază Mesajul" asincron pe fiecare bulă și un buton global "Curăță Conversația" pentru repornirea memoriei.

❖ GÂNDIRE PAS CU PAS: O interfață de chat care returnează doar text simplu, fără o reținere a memoriei de context (Array de mesaje trimis către API), este inutilă. Programează logica de stocare a stării (State) astfel încât API-ul să primească tot istoricul, nu doar ultima propoziție a utilizatorului, tratând grațios inevitabilele erori de Time-Out ale rețelei.

❖ #12

Generator Automat de Conținut AI (AI Content Generator)

Când să folosești: Atunci când dorești să dezvolți o interfață (SaaS intern sau extern) capabilă să scaleze și să automatizeze masiv producția de materiale de marketing, email-uri sau postări.

✦ **ACȚIONEAZĂ CA:** Un Arhitect de Sisteme de Automatizare (Automation Systems Architect) și Dezvoltator de Produse AI (AI Product Builder), recunoscut pentru crearea de fluxuri de lucru inteligente care se asimilează organic în departamentele de Marketing.

⬢ **OBIECTIVUL TĂU:** Construiește arhitectura completă pentru o aplicație web generativă care produce automat [specifică tipul: Postări de Social Media / Secvențe de Email / Articole de Blog / Descrieri de Produs]. Aplicația trebuie să convertească intrări simple (Inputs) într-o plajă de variante textuale finale.

❖ #12

Generator Automat de Conținut AI (AI Content Generator)

Când să folosești: Atunci când dorești să dezvolți o interfață (SaaS intern sau extern) capabilă să scaleze și să automatizeze masiv producția de materiale de marketing, email-uri sau postări.

› LIVRABIL EXECUTAT:

Structurează mașinăria de generare abordând aceste cerințe tehnice:

1. Infrastructura API-ului de Bază:
- Modelul LLM conectat: [OpenAI / Anthropic Claude / Google Gemini API]
- Câmpurile de Colectare a Datelor (User Inputs): Formular curat compus din:
- [Specifică primul input de context - ex: Subiectul postării]
- [Specifică al doilea input vizat - ex: Publicul țintă vizat]
- Selector dinamic (Dropdown) pentru setarea vocii: Tonul dorit.

2. Nucleul Promptului Intern (Hidden System Prompt):
- Coplișește AI-ul cu această instrucțiune ascunsă de sistem: "Generează simultan 3 variante complet diferite de [tipul conținutului], optimizate pentru conversie, respectând strict variabilele primite din input. Răspunsul trebuie structurat perfect, exclusiv în limba română."

3. Arhitectura Modulului de Ieșire (Output Interface):
- Randare Asincronă: Afișarea vizuală a celor 3-5 variante finale generate instant.
- Productivitate la Punctul de Acțiune (Action-Point Productivity): Fiecare variantă generată trebuie să fie însoțită imediat de propriul buton individual de "Copiere" (Copy to Clipboard).
- Buton global vizibil "Regenerează Variantele" (Re-roll).

4. Persistența Datelor (Data Layer):
- Salvarea automată și tăcută a tuturor textelor generate anterior în memoria locală a browserului (`localStorage`) pentru a crea un "Istoric al Proiectelor" (History Log).

❖ GÂNDIRE PAS CU PAS: O aplicație generativă devine profesională doar atunci când minimizează eforturile utilizatorului. Nu cere prompt-uri complexe de la utilizator, cere variabile izolate (Drop-downs, Checkboxes). Scrie logica internă astfel încât să asambleze automat aceste intrări izolate într-un "Mega-Prompt" absolut invizibil trimis către API.

❖ #13

Arhitectura unui API Backend (Node.js/Express)

Când să folosești: Atunci când ai nevoie de propriul server stabil (Backend) pentru a decupla logica de afaceri de interfața vizuală, expunând și securizând datele printr-un API propriu.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Sisteme Distribuite (Distributed Systems Engineer) și Programator Backend Senior, recunoscut pentru construirea de arhitecturi API ultra-rapide, ușoare și extrem de scalabile.

⬢ **OBIECTIVUL TĂU:** Construiește o pereche completă de Server + API de tip REST (Representational State Transfer) folosind mediul Node.js și platforma Express. Arhitectura va fi dedicată manipulării datelor pentru [specifică domeniul: Portal de Utilizatori / Catalog de Produse / Platformă de Blog].

❖ #13

Arhitectura unui API Backend (Node.js/Express)

Când să folosești: Atunci când ai nevoie de propriul server stabil (Backend) pentru a decupla logica de afaceri de interfața vizuală, expunând și securizând datele printr-un API propriu.

› LIVRABIL EXECUTAT:
Structurează codul serverului backend respectând următoarele convenții antreprenoriale:

1. Rutele de Bază (RESTful Endpoints):
- `GET /[resursa]` -> Returnează lista completă a elementelor.
- `GET /[resursa]/:id` -> Returnează detaliile izolate ale unui singur element.
- `POST /[resursa]` -> Creează și introduce o nouă înregistrare în sistem.
- `PUT /[resursa]/:id` -> Suprascrie și actualizează o înregistrare existentă.
- `DELETE /[resursa]/:id` -> Elimină definitiv o înregistrare.

2. Securizare și Interceptare (Middleware):
- Implementează protecția Cross-Origin (CORS) configurată corect.
- Injectează un parser pentru corpul cererilor (JSON Body Parser).
- Dezvoltă logica de validare strictă a formatului datelor de intrare (Input Validation).
- Activează un interceptor global de erori (Global Error Handler) pentru a preveni blocarea invizibilă a serverului (Silent Crashes).

3. Conexiunea la Baza de Date:
- Configurează integrarea completă pentru logica de stocare folosind: [Specifică soluția: SQLite / MongoDB / Postgre / In-Memory (temporar ocolind o bază reală)].

❖ GÂNDIRE PAS CU PAS: Ca senior backend, știi că mentenabilitatea este critică. Livrează codul principal (`server.js` sau `app.js`), dar generează obligatoriu și fișierul de guvernanță `package.json` cu toate dependențele necesare, plus un set clar de instrucțiuni pentru pornirea serverului în mediul local.

❖ #14

Dezvoltarea Extensiilor de Browser (Chrome Extension Build)

Când să folosești: Atunci când vrei să construiești și să distribui un produs digital super-accesibil (Software-as-a-Service ușor) care modifică nativ funcționalitatea paginilor web afișate de browser.

✦ **ACȚIONEAZĂ CA:** Un Inginer Software Front-End (Front-End Software Engineer) specializat în Ecosistemul Chromium și Arhitectura Extensiilor de Browser Web, pionier în dezvoltarea de micro-aplicații.

⬢ **OBIECTIVUL TĂU:** Creează arhitectura completă a fișierelor și codul logic necesar pentru o Extensie Chrome modernă. Scopul extensiei este de a rezolva următoarea problemă: [specifică funcționalitatea principală, ex: Cronometru de Productivitate / Salvare de Link-uri / Forțarea Modului Întunecat].

❖ #14

Dezvoltarea Extensiilor de Browser (Chrome Extension Build)

Când să folosești: Atunci când vrei să construiești și să distribui un produs digital super-accesibil (Software-as-a-Service ușor) care modifică nativ funcționalitatea paginilor web afișate de browser.

› LIVRABIL EXECUTAT:
Structurează micro-aplicația respectând riguros standardele actualizate Google:

1. Declarația de Guvernanță (Manifest):
- Generează fișierul `manifest.json` exclusiv în versiunea standardizată v3 (Manifest V3).
- Declară precis permisiunile de acces necesare (Permissions) pentru ca extensia să funcționeze pe paginile active.

2. Componentele Nucleului (Core Architecture):
- Fereastra de Interacțiune (`popup.html` + `.js` + `.css`): Creează interfața grafică pe care utilizatorul o vede apăsând iconița din colț.
- Acționarea în Pagină (`content.js`): Scrie logica de manipulare directă sau citire a structurii DOM de pe site-urile vizitate (dacă extensia o cere).
- Rularea în Fundal (`background/service_worker.js`): Implementează logica asincronă care operează independent de filele (Tabs) deschise.

3. Funcționalități Premium Integrabile:
- Stocarea preferințelor utilizatorului în memoria persistentă a browserului (`chrome.storage`), nu în `localStorage`.
- Afișarea unei insigne dinamice (Badge) cu informații de status direct peste iconița extensiei (ex: afișarea numărului de notificări).

❖ GÂNDIRE PAS CU PAS: Extensiile de browser au limite stricte de securitate și o arhitectură izolațională. Nu amesteca responsabilitățile. Livrabilul final trebuie să conțină inclusiv un set scurt de instrucțiuni clare explicând exact cum se instalează local acest pachet de fișiere utilizând modul dezvoltator (Developer Mode) în Chrome.

❖ #15

Script Python pentru Automatizarea Sarcinilor (Automation Script)

Când să folosești: Atunci când pierzi zilnic ore prețioase executând manual operațiuni repetitive pe computerul tău (ex: organizarea de fișiere, curățarea tabelelor de date, trimiterea de rapoarte) și vrei să delegi aceste sarcini unui asistent complet automatizat.

✦ **ACȚIONEAZĂ CA:** Un Inginer DevOps (DevOps Engineer) și Expert în Automatizări (Automation Specialist), recunoscut pentru eficientizarea fluxurilor de lucru corporative prin scripturi Python extrem de rapide, fiabile și cu toleranță zero la erori (Failsafe).

⬢ **OBIECTIVUL TĂU:** Construiește codul sursă complet pentru un utilitar Python conceput să automatizeze următoarea procedură: [specifică sarcina vizată: ex: Redenumirea și Clasificarea Fișierelor PDF / Backup Zilnic pe Google Drive / Procesarea a 100 de fișiere CSV]. Scrie scriptul astfel încât să poată rula invizibil în fundal, fără intervenție umană.

❖ #15

Script Python pentru Automatizarea Sarcinilor (Automation Script)

Când să folosești: Atunci când pierzi zilnic ore prețioase executând manual operațiuni repetitive pe computerul tău (ex: organizarea de fișiere, curățarea tabelelor de date, trimiterea de rapoarte) și vrei să delegi aceste sarcini unui asistent complet automatizat.

› LIVRABIL EXECUTAT:

Structurează arhitectura acestui executabil respectând următoarele exigențe la standard de producție:

1. Intrarea Datelor (Input Vectors):
- Definește clar ce date și permisiuni trebuie să primească scriptul: [ex: Calea (Path-ul) către folderul sursă și extensiile recunoscute].

2. Motorul de Execuție (Core Logic):
- Descompune logica de automatizare pas cu pas:
[Aici detaliază 2-3 pași fizici pe care scriptul îi va face cu datele]

3. Ieșiri și Audit (Output & Logging):
- Implementează un sistem robust de monitorizare (Logger) care să genereze un fișier de audit `audit_log.txt` conținând marca de timp (Timestamp) pentru fiecare acțiune și orice excepții întâlnite (Errors).
- Sistem de notificare: Alertează utilizatorul în terminal exclusiv la finalizarea cu succes sau la eșecul critic al ciclului.

4. Parametrii de Siguranță (Safety Config):
- Separarea Variabilelor: Orice date sensibile sau rute de rulare (Paths) vor fi extrase într-un fișier separat `config.json` sau `.env`.
- Modul "Dry-Run" obligatoriu: Injectează un parametru de linie de comandă (prin `argparse`) care, dacă este activat, doar va simula operațiunea (printând ce ar face) fără a modifica efectiv fișierele pe disc, pentru a preveni accidentele.

❖ GÂNDIRE PAS CU PAS: Să scrii cod care modifică și șterge fișiere local este periculos. Adoptă o programare defensivă extremă. Furnizează la final lista absolută a pachetelor externe necesare (prin Pip) și comanda exactă din terminal pentru rularea scriptului.

❖ #16

Convertor sau Calculator Utilitar Online (Web Calculators)

Când să folosești: Atunci când dorești să generezi trafic organic gratuit dezvoltând și oferind public un instrument web simplu, dar extrem de util în nișa ta (ex: Calculator de Calori/Impozit/Dobândă sau Convertor de formate).

✦ **ACȚIONEAZĂ CA:** Un Dezvoltator de Produs (Product Developer) și Inginer Front-End asertiv, obsedat de Experiența Utilizatorului (UX) și expert în livrarea micro-aplicațiilor web extrem de populare (Traffic-Magnets).

⬢ **OBIECTIVUL TĂU:** Scrie codul sursă integral (HTML, CSS și JavaScript combinat) pentru crearea unui instrument interactiv: [Specifică tipul: Calculator IMC Aprofundat / Convertor Valutar conectat la API / Generator Complex de Parole / Cronometru Pomodoro]. Instrumentul trebuie să fie instantaneu responsiv și atrăgător vizual, fără a necesita încărcarea paginii.

❖ #16

Convertor sau Calculator Utilitar Online (Web Calculators)

Când să folosești: Atunci când dorești să generezi trafic organic gratuit dezvoltând și oferind public un instrument web simplu, dar extrem de util în nișa ta (ex: Calculator de Calori/Impozit/Dobândă sau Convertor de formate).

› LIVRABIL EXECUTAT:

Structurează utilitarul conform următoarei strategii de produs tehnic:

1. Logica Matematică (Core Calculation):
- [Aici descrie formula exactă care va rula în fundal, ex: "(Greutatea în KG / Înălțimea la pătrat) x 100"]

2. Detalii Fizice de Arhitectură (Design Specs):
- Identitatea vizuală (Theme): [Minimalist / Temă Întunecată Neomorfică (Dark Neumorphism) / Corporativ]
- Complexitatea Interfeței: [Simplă (un singur flux) / Detaliată (cu opțiuni multiple avansate)]

3. Funcționalitățile Sistemului (Core Features):
- Calculare în Timp Real: Rezultatul trebuie afișat imediat și dinamic simultan cu tastarea utilizatorului (evenimentul `on-input`), eliminând necesitatea unui buton static de "Calculează".
- Scut de Validare: Câmpurile trebuie blocate la taste invalide și limitate strict la intervale logice (ex: nu se poate introduce o greutate negativă).
- Export rapid: Un buton vizibil "Copiază Rezultatul în Clipboard" lângă valoarea matematică finală.

4. Îmbunătățiri Avansate de UX (UX Enhancements):
- Istoric Sesiune: Afișarea automată, într-un panou lateral, a ultimelor 5 calcule realizate în acea vizită.
- Reacție vizuală: Implementarea unei scurte tranziții animate a numărului final de fiecare dată când rezultatul se schimbă, pentru a capta atenția vizuală.

❖ GÂNDIRE PAS CU PAS: Pe internet există milioane de calculatoare inutile. Livrează un cod absolut modular, asamblat într-un singur fișier `.html` care, la o simplă deschidere în browser, arată și funcționează ca un instrument plătit dintr-o aplicație mobilă.

❖ #17

Arhitectura unei Aplicații Mobile iOS/Android (React Native)

Când să folosești: Atunci când intenționezi să dezvoltați rapid o aplicație de mobil nativă (Mobile App) care să ruleze impecabil atât pe dispozitivele Apple (iOS), cât și pe Google (Android), dintr-o singură bază de cod.

✦ **ACȚIONEAZĂ CA:** Un Inginer Software Mobile Principal (Principal Mobile Engineer) specializat în ecosistemul React Native și dezvoltare Cross-Platform, recunoscut pentru crearea de aplicații cu 60 de cadre pe secundă (60fps) și performanță nativă perfectă.

⬢ **OBIECTIVUL TĂU:** Construiește codul complet și arhitectura de navigație pentru o nouă aplicație mobilă React Native folosind suita [Expo / React Native CLI]. Focusul principal al aplicației este: [specifică scopul, ex: Monitorizare Fitness / Livrare Mâncare / Rețea Socială].

❖ #17

Arhitectura unei Aplicații Mobile iOS/Android (React Native)

Când să folosești: Atunci când intenționezi să dezvoltați rapid o aplicație de mobil nativă (Mobile App) care să ruleze impecabil atât pe dispozitivele Apple (iOS), cât și pe Google (Android), dintr-o singură bază de cod.

› LIVRABIL EXECUTAT:
Structurează proiectul aplicației mobile respectând aceste rigori de inginerie:

1. Ecosistemul Ecranelor (Screen Architecture):
- Structura de Navigare (Routing): Implementează sistemul [React Navigation] utilizând o topologie hibridă: [ex: Navigație prin File la Bază (Bottom Tab) combinată cu Stivă de Navigare (Stack Navigation) pentru ecranele interioare].
- Ecranul 1: [Specifică funcționalitatea principală a ecranului de start]
- Ecranul 2: [Specifică funcționalitatea celui de-al doilea ecran, ex: Profilul Utilizatorului]
- Ecranul 3: [Specifică funcționalitatea de setări sau acțiune extra]

2. Componente de Înaltă Performanță (Core Components):
- Utilizează exclusiv componente native optimizate (ex: `FlatList` pentru liste lungi, NICIODATĂ `ScrollView` nativ sau `map` cu mii de rânduri).
- Injectează un spațiu de siguranță global (SafeAreaView) pentru a acomoda ecranele moderne cu notch.

3. Gestiunea Stării și a Datelor (State & Storage):
- Metodologia de gestionare a stării globale: [Context API / Redux Toolkit / Zustand]
- Soluția de stocare locală a datelor (Offline persistence): [AsyncStorage / MMKV]

❖ GÂNDIRE PAS CU PAS: O aplicație mobilă se judecă după fluiditatea ei. Scrie cod decuplat, extragând componentele de butoane și headere în fișiere separate. La finalizarea codului, imprimă instrucțiunile esențiale (`npx expo start`) și oferă un pachet mic de date simulate (Mock Data) asamblat într-un fișier extern, pentru a testa randarea interfeței instant.

❖ #18

Protocolul de Testare Automată a Codului (Automated Testing)

Când să folosești: Atunci când vrei să securizezi funcționalitatea aplicației, asigurându-te că modificările viitoare nu vor re-introduce erori vechi (Regression Bugs), facilitând lansări de produs lipsite de stres.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Asigurare a Calității Software (QA Automation Engineer) și Dezvoltator Test-Driven (TDD Practitioner), expert în construirea de plase de siguranță prin testare extensivă pentru aplicații cu milioane de utilizatori.

⬢ **OBIECTIVUL TĂU:** Analizează codul sursă de mai jos și concepe o suită exhaustivă de teste automate care să-l valideze riguros. Obiectivul tău este ca gradul de acoperire a liniilor de cod (Code Coverage) să depășească pragul de 80%.

[Lipește aici blocul curent de cod pe care vrei să-l testezi]

❖ #18

Protocolul de Testare Automată a Codului (Automated Testing)

Când să folosești: Atunci când vrei să securizezi funcționalitatea aplicației, asigurându-te că modificările viitoare nu vor re-introduce erori vechi (Regression Bugs), facilitând lansări de produs lipsite de stres.

› LIVRABIL EXECUTAT:
Structurează protocolul de testare urmărind această arhitectură defensivă:

1. Mediul de Testare (Testing Framework):
- Librăria de Assertions selectată: [ex: Jest / Vitest / Mocha / Pytest]

2. Matricea de Testare (Test Suite Types):
- Teste Unitare (Unit Tests): Izolarea și verificarea comportamentului exact al fiecărei funcții sau metode din clasă.
- Teste de Integrare (Integration Tests): Verificarea momentului în care 2 sau 3 din aceste funcții colaborează pentru a îndeplini o sarcină.
- Situațiile Limită (Edge Cases): Forțarea intenționată a funcțiilor prin injectarea de valori periculoase: `null`, `undefined`, zero, șiruri goale de text sau cifre negative.

3. Arhitectura Scenariilor (Test Architecture):
- Redactează o descriere perfect vizibilă a fiecărui bloc de cod testat prin sintaxa `describe()` și `it('should...')`.
- Implementează ciclul complet de injecție: Pregătirea datelor (Setup/BeforeEach) și Curățarea mediului (Teardown/AfterEach).
- Simulează artificial cererile către baza de date sau rețeaua externă (Mocks & Stubs) — codul de test NICIODATĂ nu atinge API-uri externe reale.

❖ GÂNDIRE PAS CU PAS: Un test inutil este mai periculos decât nicio testare deloc. Rulează întotdeauna aceste 2 scenarii obligatorii per funcție crucială: traseul ideal cu „Date valide -> Rezultat corect" (Happy Path) și traseul de siguranță „Date invalide -> Eroare capturată elegant".

❖ #19

Extragerea și Procesarea Datelor (Data Scraper)

Când să folosești: Atunci când ai nevoie să extragi, să cureți și să structurezi asincron volume imense de date de pe site-uri externe sau din documente interne, proces care altfel ar dura sute de ore manuale.

✦ **ACȚIONEAZĂ CA:** Un Inginer de Date (Data Engineer) și Expert în Extragerea Datelor Web (Web Scraping Specialist), specializat în colectarea etică, parsarea masivă și transformarea datelor brute structurate în pachete analitice (ETL).

⬢ **OBIECTIVUL TĂU:** Dezvoltă codul complet pentru un script capabil să efectueze o operațiune autonomă de extragere de date (Scraping/Processing) din următoarea sursă: [Specifică sursa datelor: URL-ul exact al paginii web / Bază de date API / Fișier local CSV].

❖ #19

Extragerea și Procesarea Datelor (Data Scraper)

Când să folosești: Atunci când ai nevoie să extragi, să cureți și să structurezi asincron volume imense de date de pe site-uri externe sau din documente interne, proces care altfel ar dura sute de ore manuale.

› LIVRABIL EXECUTAT:
Structurează logica algoritmului tău respectând aceste principii stricte de inginerie a datelor:

1. Mediul Tehnologic:
- Limbajul orientat pe execuție: [Alege: Python / Node.js]
- Ecosistemul de biblioteci (Libraries): [ex: BeautifulSoup + Requests (Python) / Puppeteer (Node) / Pandas (pentru organizare)]

2. Misiunea Extragerii (Data Extraction Map):
- Definește la nivel de selector de cod (DOM Selectors) ce informații trebuie recoltate:
- Câmpul A: [ex: Titlurile de Produs și Prețurile aferente]
- Câmpul B: [ex: Adresele de Email vizibile sau linkurile din etichetele `href`]

3. Transformare și Sanitizare (Sanitization Rules):
- Filtrarea și curățarea codului HTML rezidual (Curățarea etichetelor div/span).
- Deduplicarea automată strictă (Aruncarea informațiilor deja extrase anterior).
- Sortarea pachetului final de date pe un criteriu clar: [ex: Ordine alfabetică sau de la preț maxim la minim].

4. Livrarea și Logarea (Output Pipeline):
- Formatul documentului finit: Salvează datele finale curate transpunându-le într-un fișier `.csv` (separat prin virgulă) sau `.json`.
- Consola de Sistem: Scrie jurnale periodice (Logs) în terminal afișând progresul (ex: "50/1000 pagini procesate, fără erori").

❖ GÂNDIRE PAS CU PAS: Site-urile web își schimbă structura sau limitează accesul roboților. Scrie o logică de extragere robustă (Polite Scraping): injectează un mic decalaj (Delay) de două secunde între cereri pentru a nu supraîncărca serverul sursă și captează metodic erorile 404/403 (Error Handling) pentru a nu prăbuși complet rularea scriptului.

❖ #20

Lansarea și Distribuția Aplicației (App Deployment)

Când să folosești: Atunci când faza de scriere a codului sursă s-a terminat și vrei să lansezi aplicația pe internet public într-un mediu sigur, generând URL-ul final pe care utilizatorii îl vor putea accesa.

✦ **ACȚIONEAZĂ CA:** Un Inginer Cloud (Cloud Solutions Architect) și Specialist DevOps, obsedat de arhitecturile de lansare stabile (Deployment Pipelines) și publicarea optimizată, cu Zero Downtime.

⬢ **OBIECTIVUL TĂU:** Generează un manual tehnic detaliat și scrie fișierele de configurare necesare pentru a expune și lansa aplicația mea pe următoarea platformă de găzduire: [Vercel / Netlify / Railway / VPS Linux Personal].

❖ #20

Lansarea și Distribuția Aplicației (App Deployment)

Când să folosești: Atunci când faza de scriere a codului sursă s-a terminat și vrei să lansezi aplicația pe internet public într-un mediu sigur, generând URL-ul final pe care utilizatorii îl vor putea accesa.

› LIVRABIL EXECUTAT:

Structurează harta de lansare respectând protocolul militar pentru medii de producție (Production Environment):

1. Radiografia Arhitecturii Curente (Tech Stack Check):
- Tehnologie Frontend instalată: [React / Vue / HTML Static]
- Tehnologie Backend curentă: [Node.js / Python / Fără server (Serverless)]
- Soluție Bază de date: [PostgreSQL / MongoDB / Fără bază]

2. Fișierele de Configurare pentru Mașinăria Gazdă (Host Config):
- Scrie fișierul de rutare obligatoriu cerut de platformă (ex: `vercel.json` pentru Vercel sau `netlify.toml` pentru Netlify) fixând regulile de rescriere (Rewrites) pentru a rezolva problema erorilor 404 la actualizarea paginii (Refresh).
- Setează clar comanda de asamblare a fișierelor statice (Build Command: `npm run build`) și identifică precis folderul public final de export (Output Directory: `dist`, `build` sau `out`).

3. Securitatea și Mediul de Execuție:
- Protejarea strictă a Variabilelor de Mediu (Environment Variables) - subliniază necesitatea copierii sigure a cheilor secrete în panoul gazdei (Hosting Panel).
- Setarea permisiunilor CORS explicite pentru domeniul web pe care rulează.

4. Lista de Control Pre-Lansare (Launch Checklist):
- Verifică că Build-ul nu conține mesaje de eroare ESLint / TypeScript.
- Forțarea certificării HTTPS automată la nivel de domeniu.

❖ GÂNDIRE PAS CU PAS: Orice greșeală în faza de Deploy blochează aplicația pe internet. Ghidează-mă ca pe un junior. Spune-mi exact de ce permisiuni am nevoie pentru conexiunea dintre contul meu de GitHub și platforma de găzduire.

Începe Să Creezi

Transformă-ți Programarea Astăzi.

Inginerul Cu AI

AI

Inginerul Cu AI